

Native CTS Solver Implementation Notes

AthenaK code-reading guide

May 21, 2026

1 Purpose and Big Picture

The native initial-data solver lives primarily in `src/z4c/id_solve.cpp` and `src/z4c/id_solve.hpp`. It solves a conformal thin sandwich (CTS) system for four correction variables,

$$u = (\delta\Psi, \delta\beta^x, \delta\beta^y, \delta\beta^z),$$

stored in `IDConformalThinSandwich::u_cts`. The correction is added to a fixed background/free-data state stored in `u_free`. After the multigrid solve, the code reconstructs physical ADM variables and converts them into Z4c evolution variables.

At a high level, the flow is:

1. A problem generator initializes ADM-like seed data.
2. Z4c setup converts that seed to Z4c variables and computes initial diagnostics.
3. At the start of the Z4c task list, `Z4c_IDSolve` runs once if `pmy_pack->pid_solve` exists.
4. The CTS solver builds free data from the current ADM variables.
5. The multigrid driver solves for $\delta\Psi$ and $\delta\beta^i$.
6. `ApplySolution()` writes solved physical ADM fields, converts them to Z4c variables, refreshes boundary/ghost zones, and recomputes ADM constraints.
7. Normal hyperbolic Z4c evolution proceeds from the solved data.

2 File-by-File Map

File	Role in CTS/Z4c initial-data path
<code>src/z4c/id_solve.hpp</code>	Declares the native CTS classes and variable enums. Defines the four solved variables <code>ID_CTS_*</code> , all free-data slots <code>ID_FREE_*</code> , the <code>IDCTSMultigrid</code> subclass, the <code>IDCTSMultigridDriver</code> , and the owner class <code>IDConformalThinSandwich</code> .

File	Role in CTS/Z4c initial-data path
src/z4c/id_solve.cpp	Main implementation. Contains finite-difference helpers, conformal geometry helpers, the CTS residual operator, smoothing/defect/FAS kernels, multigrid driver construction and solve loop, free-data construction, ADM application, and constraint diagnostics.
src/z4c/z4c_tasks.cpp	Queues <code>IDConformalThinSandwich::SolveTask</code> as <code>Z4c_IDSolve</code> in the <code>Task_Start</code> phase after <code>Z4c_Recv</code> . Also shows the evolution task sequence that follows: copy, RHS, boundary RHS, RK update, restriction, send/recv, physical BCs, prolongation, algebraic constraints, ADM conversion, diagnostics, tracking, CCE, and horizon finding.
src/z4c/z4c.cpp	Constructs Z4c options and boundary buffers. Maps <code>z4c/spatial_order</code> to <code>opt.fd_stencil</code> : 2nd order $\rightarrow$ 2, 4th order $\rightarrow$ 3, 6th order $\rightarrow$ 4. Allocates <code>pbval_u</code> and <code>pbval_weyl</code> .
src/z4c/z4c_adm.cpp	Implements <code>ADMTToZ4c</code> , <code>Z4cToADM</code> , and <code>ADMConstraints</code> . This is the conversion bridge between solved ADM data and Z4c evolution arrays.
src/pgen/z4c_critical_kerr_formation.cpp	Problem generator for the compact-support conformal-metric seed. Builds $\bar{\gamma}_{ij}$, a trace-free time-derivative/extrinsic-curvature seed, zero shift, and chosen lapse; then converts ADM to Z4c before the CTS task runs.
src/multigrid/multigrid.hpp	Base multigrid data structures. Defines <code>Multigrid</code> , <code>MGOctet</code> , task IDs, storage for <code>u</code> , <code>src</code> , <code>def</code> , <code>coeff</code> , and generic restriction/prolongation/smoothing interfaces.
src/multigrid/multigrid.cpp	Generic multigrid operations: load finest data/source/coefficient arrays, restrict coefficients, retrieve solved data, restrict residuals, prolong and correct, compute defect norms, root-grid transfers.
src/multigrid/multigrid_driver.cpp	Base multigrid driver: hierarchy setup, V-cycle and FMG orchestration, root-grid boundary treatment, octet/coarse-fine bridge support, default multigrid boundary condition selection.
src/multigrid/multigrid_tasks.cpp	Task-list wiring for multigrid cycles: boundary receive/send, physical boundary fill, fine-coarse ghost fill, red/black smoothing, restriction, prolongation, and cleanup dependencies.
src/utils/finite_diff.hpp	Generated finite-difference stencils used by CTS and Z4c. The template parameter <code>NGHOST=2,3,4</code> corresponds to 2nd, 4th, and 6th-order centered operators.
inputs/z4c/gw_collapse/z4c_critical_kerr_formation.athinput	Representative input file. Sets mesh boundaries, <code>z4c/spatial_order</code> , <code>id_solve</code> controls, and the critical-Kerr seed parameters.

3 Objects and Data Arrays

3.1 Solved Variables

src/z4c/id_solve.hpp defines:

```
enum IDCTSUVar {  
    ID_CTS_PSI,  
    ID_CTS_BETAX,  
    ID_CTS_BETAY,  
    ID_CTS_BETAZ,  
    ID_CTS_NVAR  
};
```

The multigrid unknown `u_cts` stores corrections to a base conformal factor and base shift. The actual fields used by the residual are assembled with `TotalU()`:

$$\Psi = \text{ID_FREE_BASE_PSI} + u_\Psi, \quad \beta^i = \text{ID_FREE_BASE_BETA}^i + u_{\beta^i}.$$

In the current critical-Kerr seed, the base values are $\Psi = 1$ and $\beta^i = 0$, so the solver starts from no correction.

3.2 Free-Data Variables

IDCTSTFreeVar includes:

- ID_FREE_GXX ... ID_FREE_GZZ: conformal metric $\bar{\gamma}_{ij}$.
- ID_FREE_GDOTXX ... ID_FREE_GDOTZZ: contravariant trace-free tensor $\bar{u}_T^{ij}$. The names say GDOT, but the comment in the header clarifies the actual stored quantity.
- ID_FREE_K: trace K .
- ID_FREE_DKX ... ID_FREE_DKZ: $\bar{D}^i K$.
- ID_FREE_ALPHA: physical/free ADM lapse copied from `admvars.alpha` and used by the CTS operator.
- ID_FREE_E: Eulerian energy density source.
- ID_FREE_PX/PY/PZ: conformally raised momentum source $\bar{\gamma}^{ij} S_j$.
- ID_FREE_SOURCE: extra scalar source in the Hamiltonian equation.
- ID_FREE_MASK: active-cell mask for the residual and smoother.
- ID_FREE_BASE_PSI, ID_FREE_BASE_BETAX/BETAY/BETAZ: base values to which corrections are added.

4 Problem-Generator Seed Path

src/pgen/z4c_critical_kerr_formation.cpp is the current compact-support test seed. The important functions are:

- `ReadOptions()`: reads `problem/*` options such as amplitude, support radius, helicity, phase, lapse, and radial coefficients.

- `RadialProfileAndDerivative()`: constructs a compact bump profile and radial derivative using Legendre modes.
- `FillCriticalKerrFreeDataADM()`: fills ADM fields on active and ghost cells. It constructs a regular $\ell = m = 2$ solid-harmonic tensor perturbation, builds $q_{ij} = \delta_{ij} + h_{ij} + h_{ij}^2/2$, rescales it to unit determinant, and stores it as an ADM spatial metric seed. It also builds a trace-free $\dot{u}_{ij}$ -like tensor and stores $K_{ij} = 0.5 \dot{u}_{ij}$, with zero shift and configured lapse.
- `ConvertADMToZ4cAndConstraints()`: dispatches to `ADMToZ4c<2/3/4>` based on `pz4c->opt.fd_stencil`, computes ADM constraints, then calls `Z4cToADM()` so the ADM view is consistent with the Z4c storage.
- `ProblemGenerator::UserProblem()`: checks that `z4c`, `adm`, and `id_solve` exist, fills the seed, optionally sets a pre-collapsed lapse, converts to Z4c, and returns.

5 Task Scheduling and Handoff Timing

The solve is inserted in `src/z4c/z4c_tasks.cpp`. In `Z4c::QueueZ4cTasks()`, the start phase is:

```
Z4c_Recv = Z4c::InitRecv
Z4c_IRecvW = Z4c::InitRecvWeyl
Z4c_IDSolve = IDConformalThinSandwich::SolveTask, depends on Z4c_Recv
```

This means Z4c boundary receives are posted before the ID solve. The solve is still before the hyperbolic RHS and RK update. After the solve, regular evolution continues through `CopyU`, `CalcRHS`, `Z4cBoundaryRHS`, `ExpRKUpdate`, mesh communication, physical BCs, prolongation, algebraic constraint enforcement, and `Z4cToADM`.

Inside `IDConformalThinSandwich::SolveTask()`:

1. Optionally write “before” constraint diagnostics.
2. Call `IDCTSMultigridDriver::Solve()`.
3. If a solution was accepted, call `RefreshZ4cBoundariesAfterSolve()`.
4. Recompute constraints on the refreshed solution.
5. Optionally write “after” diagnostics.

The refresh step is important. `ApplySolution()` writes Z4c data, but ghost zones and physical boundaries need to be made consistent before evolution and before final diagnostics.

6 Free-Data Construction

`IDConformalThinSandwich::BuildFreeData()` converts the current ADM fields into the free-data arrays consumed by the CTS residual.

6.1 Metric, Lapse, Shift, and Trace

For every active and ghost cell:

- $\bar{\gamma}_{ij}$ is copied from `admvars.g_dd`.
- The inverse metric is computed locally.

- $K = \gamma^{ij}K_{ij}$ is computed from `admvars.vK_dd`.
- `ID_FREE_ALPHA` is copied from `admvars.alpha`. The current implementation treats this as the physical/free ADM lapse throughout the solve; it is not rescaled as a separate conformal lapse at handoff.
- The base conformal factor is set to 1.
- The base shift is copied from `admvars.beta_u`.
- The correction variables in `u_cts` are reset to zero.

6.2 Matter Sources and Masks

If `pmy_pack->ptmunu` exists, the code reads Eulerian energy density `tmunu.E` and raises the covariant momentum `S_d` using the conformal metric inverse. Otherwise $E = 0$ and $p^i = 0$. With this storage convention, the momentum residual term $\Psi^6 P^i$ corresponds to the usual $\Psi^{10} j^i$ physical-momentum source after the conformal raising is accounted for.

The mask controls where the residual and smoother are active. If `id_solve/mask_horizon_defect=true`, cells inside `horizon_mask_radius` get `ID_FREE_MASK=0`. If `id_solve/fill_horizon_junk=true`, source terms inside `horizon_fill_radius` are adjusted by `FillHorizonJunk()`.

6.3 $\bar{u}_{TF}^{ij}$ and $\bar{D}^i K$

`BuildGammaDotAndDK<NGHOST>()` computes:

- $\bar{D}^i K = \bar{\gamma}^{ij} \partial_j K$, stored in `ID_FREE_DKX/DKY/DKZ`.
- The trace-free extrinsic-curvature seed $\hat{A}^{ij}$ by raising the trace-free part of K_{ij} using the conformal metric.
- The conformal longitudinal shift operator $(\bar{L}\beta)^{ij}$.
- The stored tensor

$$\bar{u}_{TF}^{ij} = 2\alpha \hat{A}^{ij} - (\bar{L}\beta)^{ij}.$$

The halo width needed here follows the compact $D_j \hat{A}^{ij}$ path: the stored trace-free tensor must be available far enough into the ghost zone for the later centered derivative in the momentum residual.

7 CTS Residual Operator

`CTSOperator<NGHOST>()` evaluates the four residual equations at one cell. Its inputs are:

- `u`: current multigrid correction variables.
- `free`: coefficient/free-data array.
- `idx`: inverse grid spacing.
- `fd_stencil`: base derivative stencil selector.

7.1 Geometry Helpers

- `MetricInverse()` computes $\bar{\gamma}^{ij}$.
- `Christoffel<NGHOST>()` computes $\bar{\Gamma}^i_{jk}$ using centered finite differences of the conformal metric.
- `RicciScalar<NGHOST>()` computes $\bar{R}$ from first and second finite differences of $\bar{\gamma}_{ij}$.
- `DxTotal()`, `DxxTotal()`, and `DxyTotal()` differentiate base-plus-correction quantities.

7.2 $\hat{A}^{ij}$

`AHatUU<NGHOST>()` computes:

$$\hat{A}^{ij} = \frac{\bar{u}_{TF}^{ij} + (\bar{L}\beta)^{ij}}{2\alpha}.$$

`DxAHatUU()` applies the chain rule to differentiate $\hat{A}^{ij}$ directly at the cell center. The derivative is expanded into first and second finite differences of the primitive fields β^i , $\bar{\gamma}_{ij}$, $\bar{u}_{TF}^{ij}$, and α . These derivatives use the existing `src/utils/finite_diff.hpp` operators: `Dx`, `Dxx`, `Dxy`, and the base-plus-correction wrappers `DxTotal`, `DxxTotal`, and `DxyTotal`. This compact form is the only implemented $D_j \hat{A}^{ij}$ path.

7.3 Hamiltonian Residual

The implemented scalar residual is:

$$\bar{D}^2 \Psi - \frac{1}{8} \bar{R} \Psi + \frac{1}{8} \hat{A}_{ij} \hat{A}^{ij} \Psi^{-7} + 2\pi E \Psi^5 - \frac{1}{12} K^2 \Psi^5 - S = 0.$$

Here S is `ID_FREE_SOURCE`. In vacuum critical-Kerr tests, $E = S = 0$.

7.4 Momentum Residual

For each component i , the implemented residual is:

$$2\bar{D}_j \hat{A}^{ij} - \frac{4}{3} \Psi^6 \bar{D}^i K - 16\pi \Psi^6 P^i = 0.$$

$\bar{D}_j \hat{A}^{ij}$ is assembled from $\partial_j \hat{A}^{ij}$ plus Christoffel terms.

7.5 Stencil and Halo Selection

`z4c/spatial_order` maps to `fd_stencil` as:

$$2 \mapsto 2, \quad 4 \mapsto 3, \quad 6 \mapsto 4.$$

`NGHOST=2,3,4` in `finite_diff.hpp` gives 2nd, 4th, and 6th-order centered derivative formulas.

The $D_j \hat{A}^{ij}$ path is compact: it differentiates the expanded expression directly and uses the base finite-difference stencil. Because the current free-data setup computes $\bar{u}_{TF}^{ij}$ from ADM data before the solve, that prepass fills a halo of width `fd_stencil` - 1 for $\bar{u}_{TF}^{ij}$. Computing that halo also uses centered derivatives, so the free-data prepass requires enough mesh ghost cells for the prepass stencil plus the stored $\bar{u}_{TF}^{ij}$ halo. For the current 4th-order critical-Kerr tests this is `mesh/nghost=4`. In general the constructor requires

$$\text{mesh/nghost} \geq 2(\text{fd_stencil} - 1),$$

which is a CTS free-data prepass requirement. The multigrid correction-variable halo separately requires

$$\text{id_solve/mg_nghost} \geq \text{fd_stencil}.$$

The fine CTS operator order is used only on the finest MeshBlock MG level. Coarser MeshBlock levels and all root-grid levels use `id_solve/mg_coarse_fd_stencil`; the default is 2 when the fine operator is higher order. This keeps high-order accuracy on the final residual while making coarse-level FAS operators less halo hungry and less sensitive to coarse coefficient interpolation. The octet bridge still uses the independent `id_solve/octet_fd_stencil` control and remains guarded for sixth-order SMR.

8 Smoothing Kernel

The smoother is in `SmoothImpl()` and is called by `IDCTSMultigrid::SmoothPack()`. The driver reads `id_solve/smooth`; the default is the original red-black, pointwise, diagonally preconditioned nonlinear relaxation:

$$u_v \leftarrow u_v - \omega \frac{\mathcal{F}_v(u) - \text{src}_v}{D_v}.$$

Here D_v is the approximate diagonal coefficient returned by `CTSOperator()`. The scalar equation uses a diagonal built from the centered Laplacian coefficient and local nonlinear derivatives. The shift equations use a diagonal approximation to the vector Laplacian-like principal part divided by α .

This diagonal mode has the following properties:

- The update writes in-place to the live correction view and is red-black colored.
- It does not solve a coupled local 4×4 Newton system for (Ψ, β^i) .
- It ignores off-diagonal Jacobian couplings between Ψ and β^i , and between shift components.
- It is therefore closer to an under-relaxed diagonal Newton/Jacobi-GS smoother.

The optional `smoother = newton_gs` path applies a coupled local Newton update at each active red/black point. At a fixed cell, it forms the local residual

$$R_v = \mathcal{F}_v(u) - \text{src}_v$$

for $v = (\Psi, \beta^x, \beta^y, \beta^z)$, builds a scale-aware 4×4 Jacobian by finite differencing the full local CTS operator with respect to the four cell-centered unknowns, solves

$$J_{vw} \delta u_w = -R_v,$$

and applies a residual-checked damped update. The finite-difference Jacobian uses a read-only local perturbed view of the solution, so neighboring threads never observe a temporary perturbation of the shared grid. The local linear solve scales rows and columns before pivoting; if the solve is singular or ill-conditioned, the code falls back to the diagonal update at that cell.

The Newton-Gauss-Seidel option is closer to the relaxation described by East et al. (arXiv:1208.3473), because it includes the local coupling among Ψ and the three shift components. It is still damped by ω . The residual-checked line search makes it much harder for this mode to take destructive coupled Newton directions, but it remains more expensive than the diagonal smoother because it finite-differences and solves a local 4×4 system at each active point.

The smoothing update mode is now `smoother_update=frozen_view`. Before each smoothing visit, the active MG level is copied into a dedicated scratch view. `CTSOperator()`, the diagonal update, the

`newton_gs` Jacobian, and the line-search residual all read from this frozen view, while accepted updates write to the live `u` view. This removes same-color read/write races from mixed derivatives and high-order centered stencils without replacing the current two-color task structure with a much larger multi-color schedule.

Properties common to both smoother modes:

- The update is active only where `ID_FREE_MASK >= 0.5`.
- MeshBlock smoothing and root-octet smoothing use the same smoother mode.
- MeshBlock/root smoothing uses a dedicated `DualArray5D` frozen scratch view; octet smoothing uses a local frozen copy of the octet data.
- Non-finite proposed updates are rejected.
- `ngs_max_update` gives an optional absolute update cap, and `smoother_max_update_fraction` caps updates relative to the local total-field scale.
- After each accepted or fallback point update it floors Ψ at 10^{-8} .

8.1 Smoother Production Safeguards

The coupled Newton-Gauss-Seidel smoother is the more faithful algorithmic target for a nonlinear FAS CTS solve, but it should still earn default status through wall-clock validation rather than through formulation alone. The implemented production safeguards are:

1. For `newton_gs`, after solving $J\delta u = -R$, the code tries $\lambda\omega\delta u$ with $\lambda = 1, 1/2, 1/4, \dots$. It accepts the first candidate that reduces the local residual norm and preserves $\Psi > 0$.
2. If no Newton candidate is acceptable, or if the scaled 4×4 solve is singular, the point falls back to the diagonal update.
3. The local solve scales columns by the local total-field magnitude and scales rows before pivoting, so Ψ and β^i are not treated as if they had identical natural magnitudes.
4. Both smoothers use finite-value guards and optional update limiting.
5. Per-V-cycle diagnostics count accepted updates, limited updates, backtracked updates, diagonal fallbacks, singular local solves, non-finite proposals, rejected updates, Ψ floors, and maximum update. These counters are reduced across MPI ranks and printed when `show_smoother_stats=true`.

If the coupled smoother becomes consistently faster in defect reduction per wall-clock time, then it becomes a good candidate for the default. Until then, the diagonal smoother is less complete mathematically but remains the more predictable production default.

The default/used solver controls are read in `IDCTSMultigridDriver::IDCTSMultigridDriver()`:

- `id_solve/omega`: relaxation factor.
- `id_solve/smoother`: `diagonal` (default) or `newton_gs`.
- `id_solve/smoother_update`: currently `frozen_view`.
- `id_solve/ngs_iterations`: number of local Newton updates per smoothing visit when `smoother = newton_gs`.

- `id_solve/ngs_jacobian_eps`: relative perturbation used to finite-difference the local 4×4 Jacobian, scaled by the local total-field magnitude.
- `id_solve/ngs_max_update`: optional absolute limiter on each component update; 0 disables this limiter.
- `id_solve/ngs_line_search_steps`: maximum number of local Newton backtracking attempts.
- `id_solve/ngs_line_search_min`: minimum accepted line-search damping factor.
- `id_solve/smoother_max_update_fraction`: optional update cap as a fraction of the local total-field scale.
- `id_solve/show_smoothing_stats`: print MPI-reduced smoother counters after each V-cycle.
- `id_solve/npresmooth`, `id_solve/npostsmooth`: V-cycle smoothing counts.
- `id_solve/niteration`: fixed V-cycle budget when `threshold < 0`.
- `id_solve/threshold`: convergence threshold when nonnegative.
- `id_solve/full_multigrid`: optional FMG solve.
- `id_solve/keep_best_solution` and `id_solve/stop_on_defect_increase`: guards for unstable or over-relaxed runs.

9 Multigrid Solve Lifecycle

`IDCTSMultigridDriver::Solve()` does the following:

1. `PrepareForAMR()` constructs root-grid and octet bookkeeping.
2. Guards reject adaptive AMR and currently reject sixth-order CTS on SMR.
3. `owner_->BuildFreeData()` builds `u_free` and resets `u_cts`.
4. `mglevels_->LoadFinestData(owner_->u_cts, ...)` loads the correction initial guess.
5. `LoadSource(owner_->u_defect, ..., 0.0)` sets the source to zero. This makes the problem $F(u) = 0$ on the finest level; FAS modifies source arrays on coarser levels.
6. `LoadCoefficients(owner_->u_free, ...)` loads free-data coefficients into the multigrid coefficient arrays and fills their same-level and physical halos on the finest MeshBlock MG level.
7. `RestrictCoefficients()` restricts free-data arrays down the block hierarchy and fills coefficient halos after each coarse restriction. Root coefficients are filled with periodic copies or zero-gradient physical copies, not with correction-variable `mg_zerofixed` sign flips.
8. `SetupMultigrid()` initializes levels, tasks, root grids, and octets.
9. `TransferCoefficientsFromBlocksToRoot()` populates root and octet coefficient storage.
10. Initial defect norms are computed with `CalculateDefectNorm()`.
11. The code runs FMG, thresholded V-cycles, or fixed-count V-cycles.
12. If accepted, `RetrieveResult()` copies the solved correction back into `owner_->u_cts`.
13. `owner_->ApplySolution()` reconstructs ADM and Z4c data.

10 Restriction, Prolongation, and Defect Norms

`src/multigrid/multigrid.cpp` provides the generic transfer operations:

- `RestrictPack()`: computes the defect and restricts it to the coarser source. It also restricts the current solution.
- `ProlongateAndCorrectPack()`: computes a coarse-grid correction and prolongates it to the next-finer solution.
- `FMGProlongatePack()`: direct FMG prolongation.
- `CalculateDefectNorm()`: computes L1/L2/Linf norms. The current L1 and L2 paths multiply by cell volume, so the L2 norm is volume-normalized after the driver scale factor is applied.

The V-cycle task lists in `src/multigrid/multigrid_tasks.cpp` perform boundary exchange before smoothing/restriction/prolongation, then call `PhysicalBoundary()` and `FillFCBoundary()` before using ghost cells.

11 Boundary Conditions

11.1 CTS Multigrid Boundary Conditions

The base multigrid constructor in `src/multigrid/multigrid_driver.cpp` initializes multigrid boundary conditions from the mesh:

- Periodic mesh faces remain periodic.
- Every non-periodic mesh face defaults to `mg_zerofixed` in the generic base driver.

The CTS driver then overrides non-periodic faces according to `id_solve/mg_bc`. The current default is `zerograd`, so the critical- Kerr input file uses `mg_zerograd` on all six physical faces because all mesh faces are `outflow`. Set `mg_bc=zerofixed` to recover the older Dirichlet-zero correction boundary.

The physical boundary fill in `MultigridDriver::PhysicalBoundary()` and `MGRootBoundary()` implements:

- `mg_zerofixed`: odd reflection of the correction variable, approximately enforcing zero Dirichlet value at the boundary.
- `mg_zerograd`: even reflection, enforcing zero normal derivative.
- `periodic`: periodic copy.
- `mg_multipole`: multipole potential boundary support, inherited from gravity multigrid, but not currently exposed by `id_solve` input parameters.

Unlike the gravity driver, CTS currently exposes only `zerograd` and `zerofixed` through `id_solve/mg_bc`; multipole MG boundaries are not used by the CTS solve.

11.2 Coefficient and Coarse/Fine Boundaries

The free-data coefficients are loaded into multigrid coefficient arrays and restricted down levels. For SMR/octet handling:

- `TransferCoefficientsFromBlocksToRoot()` copies block coefficients to root cells or refined octets.
- Refined coefficient octets are restricted upward/downward as needed.
- Same-level octet neighbors copy boundary data.
- Coarse/fine octet boundaries use interpolation/prolongation helpers.
- Physical coefficient boundaries are filled by `ApplyCoefficientPhysicalBoundaries()` using Lagrange extrapolation for non-mask coefficients and direct copy for masks.

11.3 Z4c Boundary Conditions After Handoff

After `ApplySolution()` has written Z4c variables, the solve task refreshes Z4c boundaries:

1. `RestrictU()`
2. `SendU()`
3. `ClearSend()`
4. `ClearRecv()`
5. `RecvU()`
6. `Z4cBoundaryRHS()`
7. `ApplyPhysicalBCs()`
8. `Prolongate()`
9. `InitRecv()` again if evolution will continue

`Z4c::ApplyPhysicalBCs()` calls `pbval_u->Z4cBCs(...)` unless the mesh is strictly periodic, then optional user boundary conditions from the pgen. Thus, CTS solve boundary conditions and Z4c evolution boundary conditions are separate:

- CTS solves corrections with multigrid correction BCs.
- The resulting Z4c fields are then put through normal Z4c ghost exchange and physical BC machinery before evolution.

12 Applying the Solution

`IDConformalThinSandwich::ApplySolution()` is the handoff from elliptic variables to evolution variables.

At each active cell:

- Compute $\Psi = \text{base_psi} + \delta\Psi$, then Ψ^4 .
- Set physical ADM metric

$$\gamma_{ij} = \Psi^4 \bar{\gamma}_{ij}.$$

- Set `admvars.psi4` from $\Psi^4 \det(\bar{\gamma})^{1/3}$.
- Copy lapse from `ID_FREE_ALPHA`.
- Set physical shift

$$\beta^i = \beta_{\text{base}}^i + \delta\beta^i.$$

- Recompute $\hat{A}^{ij}$ from the solved shift.
- Lower $\hat{A}^{ij}$ with the conformal metric and reconstruct

$$K_{ij} = \Psi^{-2} \hat{A}_{ij} + \frac{1}{3} \gamma_{ij} K.$$

Then `ApplySolution()` calls:

1. `ADMToZ4c<fd>()`: populate Z4c evolution variables.
2. `Z4cToADM()`: make the ADM diagnostic view consistent with Z4c.
3. `ADMConstraints<fd>()`: compute constraint diagnostics on the active region.

Finally, `SolveTask()` performs the boundary refresh described above and recomputes constraints one more time on the boundary-consistent state.

13 Current Critical-Kerr Input Choices

In `inputs/z4c/gw_collapse/z4c_critical_kerr_formation.athinput`, the current representative setup is:

- Mesh BCs: `outflow` on all faces.
- ADM dynamic: `true`.
- Z4c spatial order: 4, so `fd_stencil=3`.
- CTS enabled and `solve-once`.
- `omega=0.02` in the file, though convergence sweeps found larger values useful for short second-order diagonal-smoother runs.
- `smoother=diagonal` by default. Set `smoother=newton_gs` to use the coupled local Newton-Gauss-Seidel relaxation.
- `smoother_update=frozen_view`: read operator values from a scratch copy during each smoothing visit and write updates to the live correction view.
- `ngs_line_search_steps=8`, `ngs_line_search_min=1.0e-4`, and `smoother_max_update_fraction=0.25`: local update safeguards.
- `threshold=-1.0`: fixed V-cycle count mode.
- `niteration=1` in the input file; convergence tests override this.
- `npresmooth=2`, `npostsmooth=2`.
- `show_smoother_stats=true`: print per-V-cycle smoother counters.

- `mg_nghost=4`: order-matched for current 4th-order critical-Kerr CTS tests. For 6th order with the current free-data prepass, use `mesh/nghost=6` and `id_solve/mg_nghost=6`.
- `mg_coarse_fd_stencil=2`: use the second-order CTS operator on coarse MeshBlock/root levels while keeping the fine-grid operator at the configured Z4c order.
- `mg_bc=zerograd`: use zero-normal-gradient MG boundaries for CTS correction variables on non-periodic faces.
- `reject_worse=false, keep_best_solution=true`.

14 Current Smoother Validation Snapshot

The production-smoother validation runs are recorded under `analysis/critical_kerr_convergence/smoother_comparison`. All runs used MPI with four ranks or fewer. The current snapshot is:

- At 32^3 , second order, 10 V-cycles, the diagonal smoother was stable for $\omega = 0.02, 0.05, 0.08$. The new residual-checked `newton_gs` path was stable for $\omega = 0.005, 0.02, 0.05$.
- At 64^3 , second order, 40 V-cycles, `diagonal`, $\omega = 0.05$, reduced the total CTS defect by a factor 7.10×10^{-2} in about 152 seconds.
- The matching `newton_gs`, $\omega = 0.05$, run reduced the total CTS defect by a factor 1.46×10^{-2} , but took about 525 seconds. It therefore reached a lower final defect but was slower in defect reduction per wall-clock time for this case.
- At 32^3 , fourth order with `nghost=4`, both smoothers completed the short 5-cycle smoke test without non-finite updates.
- A short `mg_bc=zerofixed` comparison also remained bounded, though `mg_bc=zerograd` remains the less intrusive default.

The smoother counters showed no singular local solves, non-finite proposals, fallbacks, or Ψ floors in these validation cases. That means the new safeguards are not masking a frequent failure in this regime, but they remain important for stronger free data, coarser grids, and FAS coarse-level states.

15 Known Implementation Caveats

- The default smoother remains the diagonal approximate Newton smoother for continuity with previous runs. The optional `newton_gs` smoother now has local residual-checked globalization and scale-aware linear algebra, but it is still more expensive because it builds a finite-difference Jacobian at every active point.
- Frozen-view smoothing removes same-color races for the current stencil family. It may be a weaker smoother than a fully stencil-aware multi-color Gauss-Seidel schedule, so multi-color remains a possible future optimization if frozen-view convergence is too slow.
- The matter-source convention in the residual uses $E\Psi^5$ and $P^i\Psi^6$, where $P^i = \bar{\gamma}^{ij}S_j$ is the stored conformally raised momentum source. In vacuum this is irrelevant, but matter runs should preserve this convention consistently.
- `ID_FREE_ALPHA` is currently the physical/free ADM lapse used consistently by `BuildFreeData()`, `CTSOperator()`, and `ApplySolution()`. A future conformal-lapse convention would need explicit conversion both when building free data and when applying the solution.

- The CTS driver exposes `id_solve/mg_bc=zero|zerofixed`. This controls only the CTS multigrid correction-variable boundary; after handoff, Z4c fields still use the normal Z4c/mesh boundary machinery.
- The compact $D_j \hat{A}^{ij}$ path should be validated by convergence tests whenever changing order, boundary conditions, or free data.
- Coarse MeshBlock/root levels intentionally default to lower-order CTS operators. This should not limit fine-grid convergence if the fine residual is solved tightly, but it should be included in validation reports.
- Sixth-order CTS on SMR is explicitly guarded off because the current octet coarse/fine bridge is not widened enough for the required stencil.

16 Where to Start Reading

A good code-reading order is:

1. `src/z4c/id_solve.hpp`: understand variable enums and class layout.
2. `src/pgen/z4c_critical_kerr_formation.cpp`: see how the seed data are constructed.
3. `src/z4c/z4c_tasks.cpp`: see where the solve is queued.
4. `src/z4c/id_solve.cpp`, starting with: `IDConformalThinSandwich::SolveTask()`, `IDCTSMultigridDriver::BuildFreeData()`, `CTSOperator()`, `SmoothImpl()`, and `ApplySolution()`.
5. `src/multigrid/multigrid_driver.cpp` and `src/multigrid/multigrid_tasks.cpp`: see how V-cycles, red/black smoothing, and boundary exchanges are scheduled.
6. `src/z4c/z4c_adm.cpp`: see the ADM/Z4c conversion and constraint diagnostics.